BEHAVIOUR-AWARE PORTFOLIO CONSTRUCTION

How much of an investor's money should go into HUL, Sun Pharma and ICICI Bank, given how much risk that investor can bear?

Abhay S (246213044)
Sankarshana Mulugundam (246213080)

DATA FROZEN ON 13 OCT 2026

ONE FILE: PYTHON_PROJECT.PY

1,108
LINES OF PYTHON

14
NUMBERED SECTIONS

21
AUTOMATED TESTS AGAINST THE REAL PIPELINE AND EXCEL

FROM ANSWERS AND PRICES TO RUPEES

The input is ten questionnaire answers and an amount. The output is a rupee split and a measure of risk.

PRICES AND RETURNS
Daily, Jan 2016 to 13 Oct 2026: Parquet files built with yfinance

VALUATION INPUTS
workbook/valuation.xlsx, read cell by cell with openpyxl

SETTINGS
config.yaml: risk-free rate, simulation sizes, SIP terms

TEXT
2,428 headlines scored by FinBERT; PDFs read with pypdf

EXAMPLE OUTPUT

A = 4

BALANCED INVESTOR, ₹1,00,000

HUL	**₹13,382**
Sun Pharma	**₹8,394**
ICICI Bank	**₹21,833**
T-bills	**₹56,392**

One-year 95% VaR: ₹5,582

FOUR STAGES, AS PLANNED IN OUR PROPOSAL

Each stage shaped how the file is organised.

DECOMPOSITION

Split one question into smaller ones: load and validate data; measure risk; value each stock; score the investor; optimise; simulate; report.

IN THE CODE

14 sections, each one job

PATTERN RECOGNITION

Volatility, correlation and beta in returns. Every valuation model has the same shape: cash flows plus a terminal value.

IN THE CODE

The shared base class

ABSTRACTION

Keep only what the decision needs. An Investor is a name, A, an amount and a SIP. A Security is a beta, a return and a volatility.

IN THE CODE

Three small classes

ALGORITHM DESIGN

Recursive bisection for implied growth; merge sort and binary search for outcomes; Cholesky for correlated simulation.

IN THE CODE

Each checked another way

EVERY COURSE CONCEPT, MAPPED TO THE FILE

The same map sits at the top of python_project.py. Each concept does real work in the project.

Concept	Section	What it does in the project
Variables, numbers, strings	1	Settings; Indian-style rupee formatting
Files and exceptions	2	JSON, YAML, CSV, Parquet, PDF; a clear DataError
Functions, Pandas	3	Returns, Sharpe, beta, Treynor, Jensen's alpha
Decisions, loops, dictionaries	4, 10	Questionnaire scoring; trading-rule signals
Classes	5	Security, Investor, Portfolio
Inheritance, recursion	6	ValuationModel and two subclasses; recursive PV, bisection
Sorting and searching	7	Merge sort, binary search, grid search
SciPy	8	Maximum Sharpe weights, efficient frontier
NumPy	9, 10	Cholesky Monte Carlo, block bootstrap, EWMA, SIP
Sets, Transformers, pypdf, yfinance	2, 11	Hedge words, FinBERT, PDF text, price download
Matplotlib, Seaborn	12	Frontier, fan chart, correlation heatmap
Streamlit	14	A small interactive demo in the same file

ONE BASE MODEL, TWO SUBCLASSES

The base class holds discounting and the reverse solve. A subclass only says what its cash flows are.

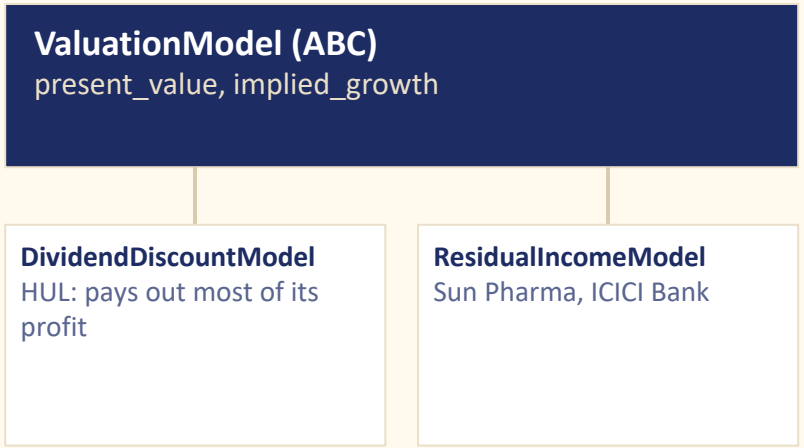
```
class DividendDiscountModel(ValuationModel):

    def __init__(self, name, price, r, g2, n, d0: float, g1: float):
        super().__init__(name, price, r, g2, n)
        self.d0, self.g1 = d0, g1

    def cash_flows(self) -> list[float]:
        flows, dividend = [], self.d0
        for year in range(1, self.HORIZON + 1):
            dividend *= 1 + (self.g1 if year <= self.n else self.g2)
            flows.append(dividend)
        return flows

    def terminal_value(self) -> float:
        return self.cash_flows()[-1] * (1 + self.g2) / (self.r - self.g2)
```

present_value and implied_growth are not written here: both come from the base class.



BUSINESS ENTITIES

Security (dataclass): ticker, beta, return, volatility
Investor: A, amount, SIP; from_answers classmethod
Portfolio: checks weights sum to 1; complete, allocation, ranked

TWO RECURSIVE FUNCTIONS, EACH CHECKED BY ALGEBRA

Each has a base case and a smaller problem. Each result is compared with a closed-form formula and the Excel workbook.

```
def present_value(flows: list[float], r: float, t: int = 1) -> float:
    if not flows:           # base case: nothing left to discount
        return 0.0
    return flows[0] / (1 + r) ** t + ValuationModel.present_value(flows[1:], r, t + 1)

def implied_growth(self, low, high, tol):
    gap_low = self.single_stage_value(low) - self.price
    middle = (low + high) / 2
    if high - low < tol:    # base case: the range is small enough
        return middle
    if gap_low * (self.single_stage_value(middle) - self.price) <= 0:
        return self.implied_growth(low, middle, tol)
    return self.implied_growth(middle, high, tol)
```

Stock	Class	Value	Implied growth
HUL	Dividend	₹1,079.12	7.41%
Sun Pharma	Residual income	₹747.63	8.63%
ICICI Bank	Residual income	₹574.65	13.89%

BASE CASE

An empty list of cash flows is worth 0. For bisection, a range narrower than 1e-10 is the answer.

SMALLER PROBLEM

One fewer cash flow, one year later. Bisection halves its range on every call.

PROOF

A test asserts recursive value = closed form = Excel, and bisection = algebra.

BAD DATA STOPS THE PROGRAM, WITH A CLEAR MESSAGE

We never fill a gap with a guess. A missing file or column raises DataError, which names the file and the fix.

```
def read_returns(path: Path, needed: list[str]) -> pd.DataFrame:
    try:
        table = pd.read_parquet(path)
    except FileNotFoundError as err:
        raise DataError(f"{path} not found; run the pipeline first") from err
    missing = [c for c in needed if c not in table.columns]
    if missing:
        raise DataError(f"{path} lacks columns {missing}")
    gaps = int(table[needed].isna().sum().sum())
    if gaps:
        raise DataError(f"{path} has {gaps} missing values in {needed}")
    return table[needed]
```

DICTIONARY
QUESTIONS maps each question to its kind, text and five options

LIST
The answers for capacity and for willingness, kept separately

SET
HEDGE_WORDS: fast lookup of cautious words like may, might, could

A missing file is caught, not ignored: `...does_not_exist.parquet not found; run the pipeline first`
Bad inputs are refused: `Bad input: the discount rate 5.00% must exceed terminal growth 6.00%`

10,000 SIMULATED YEARS, WITHOUT A PYTHON LOOP

Monte Carlo: Cholesky keeps the correlation between the stocks; arrays do the work on every path at once.

```
def simulate_normal(mu, cov, n_sims, horizon, rng):
    chol = np.linalg.cholesky(cov)
    return rng.standard_normal((n_sims, horizon, len(mu))) @ chol.T + mu

def wealth_paths(risky_returns, y, rf_period):
    growth = np.cumprod(1 + y * risky_returns + (1 - y) * rf_period, axis=1)
    return np.hstack([np.ones((risky_returns.shape[0], 1)), growth])
```

₹1,08,463

MEDIAN AFTER ONE YEAR, ₹1 LAKH, A = 4

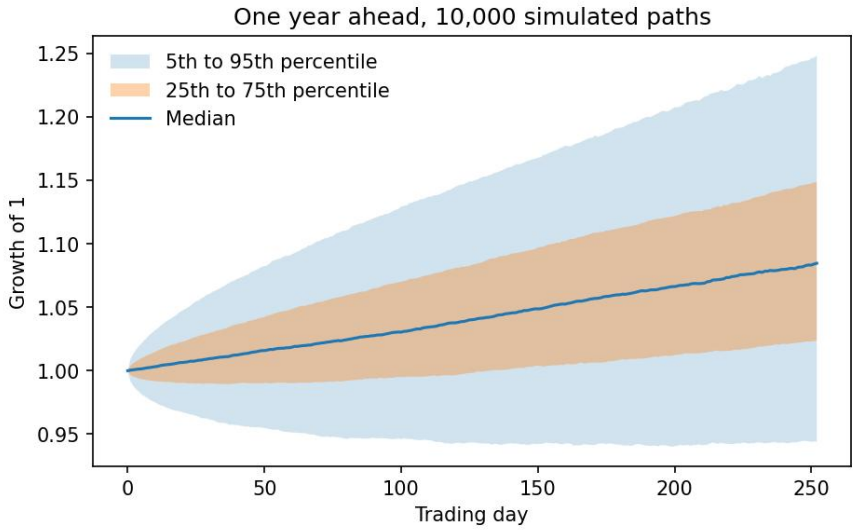
₹5,582

95% VaR, NORMAL RETURNS

₹5,465

95% VaR, BLOCK BOOTSTRAP

Cholesky: split the covariance matrix so that cov = L L^T. Independent draws multiplied by L have the correlations we measured.



Half of all paths end between ₹1,02,377 and ₹1,14,860.

Type hints trimmed to fit the slide.

THE OPTIMISER, CHECKED BY BRUTE FORCE

SciPy finds the maximum-Sharpe weights within 10–60% limits. A separate grid search on every 1% mix finds the same portfolio.

```
def max_sharpe_weights(mu, cov, rf: float, bounds=WEIGHT_BOUNDS) -> np.ndarray:
    mu, cov = np.asarray(mu), np.asarray(cov)
    return _optimise(lambda w: -(w @ mu - rf) / np.sqrt(w @ cov @ w), len(mu), bounds)

def _optimise(objective, n: int, bounds, constraints=()) -> np.ndarray:
    all_constraints = [{"type": "eq", "fun": lambda w: w.sum() - 1.0}, *constraints]
    result = minimize(objective, np.full(n, 1.0 / n), method="SLSQP", bounds=[bounds] * n,
                      constraints=all_constraints, options={"ftol": 1e-12, "maxiter": 500})
```

SCIPY SLSQP

31 / 19 / 50

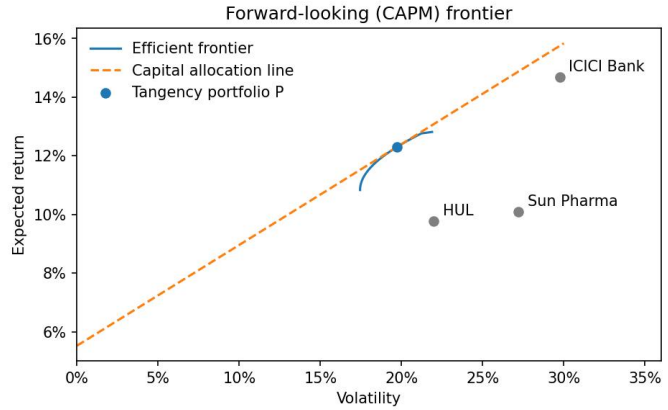
Sharpe 0.3436

1% GRID SEARCH

31 / 19 / 50

Same portfolio, found by trying every mix

Then $y^* = (E[rP] - rf) / (A\sigma^2)$ sizes it for each investor: 87% at A = 2, 44% at A = 4, 29% at A = 6. The rest goes to T-bills.



RANKING 10,000 OUTCOMES, FINDING WHAT MATTERS

Merge sort ranks the outcomes. Binary search then finds how many end below ₹1 lakh.

```
def merge_sort(items: list, key=lambda x: x) -> list:
    if len(items) <= 1:
        return list(items)
    middle = len(items) // 2
    left, right = merge_sort(items[:middle], key), merge_sort(items[middle:], key)
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    return merged + left[i:] + right[j:]
```

```
def binary_search(ordered, value):
    low, high = 0, len(ordered)
    while low < high:
        middle = (low + high) // 2
        if ordered[middle] < value:
            low = middle + 1
        else:
            high = middle
    return low
```

Each step halves the list: about 14 steps. Scanning it could take 10,000.

WORST THREE

₹78,719, ₹79,877, ₹80,171

BEST THREE

₹1,46,320, ₹1,46,757, ₹1,47,604

BELOW ₹1 LAKH

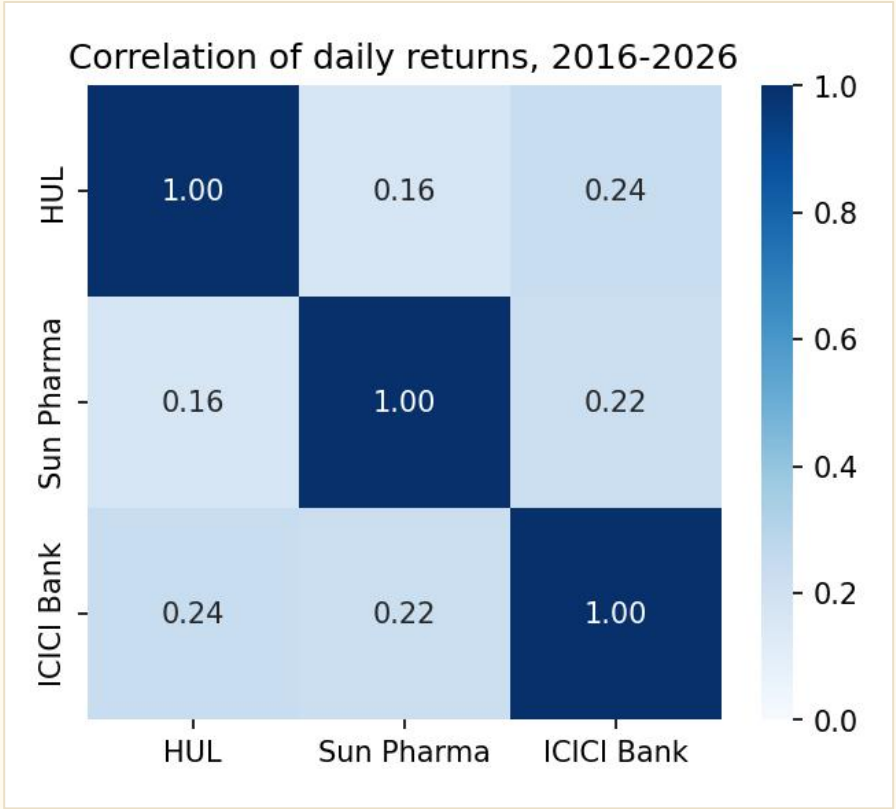
17.1% of paths

Merge sort checked against sorted() on 2,000 paths; sorted() ranks all 10,000. A = 4, ₹1 lakh, one year. Type hints trimmed.

EACH LIBRARY HAS ONE JOB

Heavy libraries are imported inside the function that needs them, so the file still starts on a machine that lacks one.

Library	Job
Pandas	Load, align and summarise returns; metrics tables
Matplotlib	Efficient frontier and Monte Carlo fan chart
Seaborn	Correlation heatmap
yfinance	Fresh prices (--fetch)
Transformers (FinBERT)	Financial tone of a sentence (--finbert)
pypdf	Text from annual reports and transcripts (--pdf)
openpyxl	Excel valuation inputs, cell by cell
Streamlit	Interactive demo on one screen
requests	Optional live answers through OpenRouter



THE SAME PORTFOLIO, SIZED THREE WAYS

On ₹1 lakh for one year, then a ₹10,000 monthly SIP rising 10% a year for 15 years. Simulated; not investment advice.

AGGRESSIVE · A = 2

87%

IN THE THREE STOCKS

Expected return	**11.4%**
Volatility	**17.2%**
95% VaR on ₹1 lakh	**₹16,138**
Chance of loss	**28%**
15-year SIP median	**₹69.5 lakh**

BALANCED · A = 4

44%

IN THE THREE STOCKS

Expected return	**8.5%**
Volatility	**8.6%**
95% VaR on ₹1 lakh	**₹5,582**
Chance of loss	**17%**
15-year SIP median	**₹64.8 lakh**

CONSERVATIVE · A = 6

29%

IN THE THREE STOCKS

Expected return	**7.5%**
Volatility	**5.7%**
95% VaR on ₹1 lakh	**₹1,940**
Chance of loss	**10%**
15-year SIP median	**₹61.3 lakh**

21 TESTS: HOW WE KNOW THE CODE IS RIGHT

Because this file is self-contained, it could quietly drift from the rest of the project. The tests stop that.

SAME ANSWERS

- Valuations equal the Excel workbook
- Metrics and weights equal the pipeline's files
- Simulations reproduce mc.json and sip.json exactly
- Trading rules match the pipeline's

SECOND METHODS

- Recursion and bisection equal closed forms
- Grid search finds SciPy's optimum
- Own merge sort equals Python's sorted
- Questionnaire equals the dashboard's

BAD INPUTS

- Discount rate below growth is refused
- Weights not summing to 1 are refused
- A blank Excel cell gives a clear error
- Missing and broken files raise DataError

```
python -m pytest tests/test_python_project.py    21 passed
```

FOUR COMMANDS

Everything runs on the frozen data, so the screen matches these slides.

01	`python python_project.py`	The full tour: each section prints its result next to the pipeline's, marked OK
02	`python python_project.py --charts out/`	Saves the Matplotlib and Seaborn charts
03	`streamlit run python_project.py`	Move A from 4 to 2: the share in P goes from 44% to 87%
04	`python -m pytest tests/test_python_project.py`	21 tests, in a few seconds

IF A COMMAND FAILS Show the saved output and charts in deliverables/assets/python, and keep going.

AI ASSISTANCE, AND WHAT CHANGED FROM THE PROPOSAL

We list what differs from the approved proposal, and why, so nothing is hidden.

Proposal said	The project does	Why
FINRA NCFS investor microdata	A 10-question questionnaire (capacity and willingness)	It sets A for one investor directly; we did not calibrate it on NCFS
Loughran–McDonald word lists	A small hedge-word set of the same kind	Keeps the file self-contained; it is not the full list
NSE total-return index	Nifty 50 price plus an estimated 1.22% a year	Estimated from the gap between Nifty BeES and the index
Anthropic SDK (optional)	A free model through OpenRouter (optional)	No cost for a live demo; every number still comes from our files

AI ASSISTANCE

Claude helped write and review this code. We checked it by running it, by testing it against Excel and the pipeline, and by reading it until we could explain each function; docs/python_walkthrough.md is our map. This is coursework, not investment advice.